Universidad Autónoma de Madrid

Departamento de Informática

Proyecto de Sistemas Informáticos
Práctica - 3

Roberto Marabini

Registro de Cambios

Versión[1]	**Fecha**	**Autor**	**Descripción**
1.0	12.07.2025	RM	Primera versión.
1.1	12.09.2025	RM	Corregir URLs y añadir criterios de corrección
1.2	26.09.2025	RM	Revisión
1.3	15.2.2026	RM	Revisión previa al comienzo de las prácticas para clarificar los requerimientos solicitados

[1]La asignación de versiones se realizan mediante 2 números $X.Y$. Cambios en Y indican aclaraciones, descripciones más detalladas de algún punto o traducciones. Cambios en X indican modificaciones más profundas que o bien varían el material suministrado o el contenido de la práctica.

Índice

1. Objetivo **3**
1.1. Sistema de control de versiones: `git` 5
1.2. Automatización con Makefile 5

2. Implementación del modelo de datos (tiempo sugerido: la mitad de la práctica) **6**
2.1. Test 11

3. Implementación del API REST (tiempo sugerido: la mitad de la práctica) **13**
3.1. Usuarios 13
3.2. Test 15
3.3. Otros Endpoints 15
3.4. Testing y Cobertura 17

4. Despliegue de la aplicación **17**

5. Test ejecutable en render.com **18**

6. Resumen del trabajo a entregar **18**

7. Criterios de evaluación **19**

A. Song **21**

1. Objetivo

En esta práctica se propone el desarrollo de una aplicación web destinada al aprendizaje de idiomas a través de la música, aprovechando el potencial pedagógico de las canciones como recurso didáctico.

La aplicación estará inspirada en plataformas como *LyricsGap* o *LyricsTraining*, que utilizan letras de canciones para reforzar la comprensión auditiva y el vocabulario. En particular, se implementará una plataforma web que permita al usuario escuchar una canción y completar palabras ausentes en la letra.

La práctica incluirá el diseño de la interfaz de usuario, la integración de audio y letras sincronizadas, y la lógica para la detección y validación de respuestas del usuario.

Herramientas Tecnológicas y Estructura del Proyecto

Desde una perspectiva técnica, la aplicación se desarrollará utilizando una arquitectura basada en la separación entre el backend y el frontend. El backend de la aplicación estará implementado con el framework *Django*. El backend será responsable de gestionar los usuarios, almacenar las letras de canciones, registrar el progreso de los estudiantes y servir los datos necesarios al frontend mediante una API REST. El frontend se desarrollará utilizando *Vue.js*. La comunicación entre el frontend y el backend se realizará mediante peticiones HTTP a la API REST desarrollada en *Django*.

En las URLs `https://song-54se.onrender.com`/api/v1/ (backend) y `https://song-frontend-shl2.onrender.com` (frontend) se puede consultar la implementación preliminar que hemos realizado de esta aplicación como preparación para la redacción de esta práctica. Podéis usar esta implementación como referencia pero no se espera, ni se desea, que realicéis una copia de la misma.

Requerimientos del Sistema

Requerimientos Funcionales

1. El sistema debe permitir el inicio de sesión y cierre de sesión de usuarios. NO se requiere que el sistema gestione la creación de usuarios.
2. NO se requiere que el interfaz gestione la subida de canciones con sus letras y archivos de audio.
3. Las letras deben poder editarse para ocultar palabras específicas que serán usadas en los ejercicios.
4. El usuario podrá reproducir canciones mientras visualiza la letra sincronizada.
5. Se debe validar en tiempo real las respuestas del usuario y proporcionar retroalimentación.
6. El sistema debe registrar el progreso del usuario y mostrar estadísticas de desempeño.
7. El usuario podrá buscar canciones por título.
8. El sistema NO soportará canciones tipo "canon", esto es, canciones en las que hay varias voces en paralelo.

Requerimientos No Funcionales

- **Seguridad:** Las contraseñas deben almacenarse cifradas y las APIs deben estar protegidas contra accesos no autorizados. .

El código debe

1. Satisfacer los criterios de estilo marcados por las utilidades *Flake8* (python) y `npm run lint` (vue.js).
2. Usar las versiones de los módulos *Django* y *Vue.js* solicitas más adelante en esta práctica.

3. Almacenar la información en una base de datos creada con el gestor de bases de datos *PostgreSQL*.

4. Contener todos los ficheros auxiliares necesarios para desplegar la aplicación en *Render.com*.

5. Las URLs en *Render.com* deben seguir el formato: `http://practica\_proyecto\_pareja\_grupo\_agno\_version.onrender.com` (en este caso: práctica = 3, proyecto = *songproject*, etc. `version` es una cadena arbitraria que el alumno puede elegir libremente.

1.1. Sistema de control de versiones: `git`

Durante el desarrollo de la práctica, se deberá utilizar *git* como herramienta de control de versiones. Como parte del material a entregar, se incluirá el repositorio utilizado, el cual debe reflejar una participación activa y equilibrada por parte de todos los integrantes del equipo.

Es obligatorio que el repositorio contenga *commits* frecuentes realizados por los dos miembros. En concreto, se exige un mínimo de **un commit semanal por cada integrante**. Esta medida tiene como objetivo fomentar una colaboración continua y equitativa.

Además, el repositorio deberá ser configurado como **privado**. Cualquier pareja que cree un repositorio público será automáticamente sancionada con la suspensión de la práctica, sin posibilidad de revisión.

1.2. Automatización con Makefile

Durante el desarrollo de la práctica, es habitual ejecutar repetidamente ciertos comandos. Para facilitar y automatizar estas tareas, se recomienda el uso del comando `make` junto con el archivo `Makefile` proporcionado en el material de la práctica.

Este archivo contiene una serie de reglas predefinidas que permiten ejecutar operaciones comunes de forma sencilla. A continuación, se describen algunas de las más relevantes:

- `create_super_user`: Crea un usuario con permisos de administración. El nombre de usuario y la contraseña por defecto son `alumnodb`. Equivale a ejecutar `python3 manage.py createsuperuser`.
- `runserver`: Inicia el servidor de desarrollo en el puerto 8001. Equivale a `python3 manage.py runserver 8001`.
- `force_update_db`: Elimina la base de datos, la recrea y aplica las migraciones necesarias. Ejecuta internamente los comandos `python3 manage.py makemigrations` y `python3 manage.py migrate`.
- `dbshell`: Abre el cliente de base de datos `psql` para PostgreSQL mediante `python3 manage.py dbshell`.
- `shell`: Lanza una consola interactiva de Python con el entorno de Django cargado. Equivale a `python3 manage.py shell`.

2. Implementación del modelo de datos (tiempo sugerido: la mitad de la práctica)

En esta práctica nos concentraremos en crear toda la infraestructura necesaria para implementar el servidor (*API REST*). En la práctica 4 se implementará el cliente (*Vue.js*). Durante las dos primeras semanas nos centraremos en el modelo de datos y en las 2 finales implementaremos el API REST propiamente dicho.

Para comenzar, cread un entorno virtual de Python e instalad todos los módulos enumerados en el archivo `requirements.txt` (disponible en *Moodle*). Luego, será necesario crear un proyecto en *Django* con el nombre *songproject* y, utilizando *PostgreSQL*, cread una base de datos llamada *songdatabase*. Modifica el archivo `settings.py` para que utilice esta base de datos. Asegúrate de no incluir información sensible en `settings.py`, como las claves de acceso a la base de datos o la variable `SECRET_KEY`. Finalmente, crea una aplicación llamada `song_models` e implementa en ella el siguiente modelo.

Modelo `Song`

Guarda información sobre la canción.

- `title`: `cadena` - Título de la canción, tamaño máximo 255 caracteres.
- `artist`: `cadena` - Título de la canción, tamaño máximo 255 caracteres.
- `language`: `cadena` - A elegir entre los proporcionados por la variable `LANGUAGE_CHOICES` (véase apéndice A), tamaño máximo 2 caracteres.
- `audio_file`: `nombre_fichero(upload_to='media/')` - Archivo de audio.
- `lrc_file`: `nombre_fichero(upload_to='media/')` - Archivo LRC con letra sincronizada
- `background_image`: `nombre_fichero(upload_to='media/')` - Imagen de fondo.
- `created_at`: `fecha_con_hora` - Fecha de creación del registro, por defecto momento de creación.
- `category`: `cadena` - A elegir entre los proporcionados por la variable `CATEGORY_CHOICES` (véase apéndice A), tamaño máximo 20 caracteres.
- `number_times_played`: `integer (default=0)` - número de veces que una canción ha sido reproducida por un usuario registrado y por tanto su reproducción ha quedado registrada mediante un objeto `SongUser` (véase modelo siguiente).

Modelo `SongUser`

Guarda información sobre las canciones utilizadas por un usuario. Si un mismo usuario accede varias veces a la misma canción sólo se guarda la información de la última ejecución perdiéndose los datos de ejecuciones anteriores.

- `song`: `Clave extranjera(Song, on_delete=models.CASCADE)` - Canción asociada.

- `user`: `Clave extranjera(User, on_delete=models.CASCADE)` - Usuario asociado. Se usará el model "User" implementado en `django.contrib.auth.models`
- `played_at`: `fecha_con_hora` - Fecha de creación del registro, por defecto momento de creación.
- `correct_guesses`: `Integer (default=0)` - Número de aciertos.
- `wrong_guesses`: `Integer (default=0)` - Número de errores.

Script de Población de Datos: `populate.py`

Como parte del desarrollo de la práctica, se solicita la implementación de un comando personalizado de *Django* que permita poblar la base de datos con datos de prueba. Este comando facilitará la validación del funcionamiento de la aplicación y la demostración de sus funcionalidades.

Ubicación del script

El script debe ubicarse en el archivo:

`song_models/management/commands/populate.py`

Funcionalidad esperada

El comando debe realizar las siguientes acciones:

1. Eliminar todos los registros existentes en los modelos `Song`, `SongUser` y `User`.
2. Crear al menos dos usuarios de prueba.
3. Crear un superusuario con username y password `alumnodb`.
4. Insertar varias canciones de ejemplo, incluyendo título, artista, idioma, archivo de audio, archivo LRC, una imagen de fondo, categoría.
5. Insertar varios objetos `songUser`

6. Mostrar mensajes informativos en consola durante la ejecución del comando.

En el repositorio con ficheros auxiliares que se haya linkado en *Moodle*, en el directorio `Media` podéis encontrar 3 canciones. Vuestra implementación de `populate` debe cargar al menos estas tres canciones

Ejecución del comando

Una vez implementado, el script podrá ejecutarse desde la línea de comandos con:

```
python manage.py populate
```

Este comando es útil tanto para pruebas locales como para la evaluación del proyecto, ya que permite inicializar rápidamente un entorno funcional con datos representativos.

IMPORTANTE: puebla el directorio `Media` con los ficheros de audio y las letras a las que haga referencia `populate.py`.

Ejemplo orientativo de implementación

A continuación se muestra un fragmento de código que ilustra cómo podría comenzar la implementación del comando `populate.py`. Este ejemplo no está completo, pero proporciona una guía sobre la estructura general del script:

Listado 1: template para el fichero `song_models/management/commands/populate.py`

```
# Script que:
# Elimina los datos previos almacenados en la base de datos
# Crea categorias, canciones y usuarios.
# Puebla la tabla intermedia SongUser

# BaseCommand: Clase base para crear comandos personalizados
# que se ejecutan con python manage.py <comando>.
from django.core.management.base import BaseCommand
```

```
from song_models.models import Song, Category

# User: Modelo de usuario por defecto de Django.
from django.contrib.auth.models import User


class Command(BaseCommand):
    help = 'Populate the database with sample data'

    # Este metodo se ejecuta cuando llamas al comando.
    # Aqui se implementa la logica para crear datos.
    def handle(self, *args, **kwargs):
        # Se eliminan registros previos para evitar duplicados.
        SongUser.objects.all().delete()
        # borrar otros modelos
        ...


        # Crear canciones (completar con datos de ejemplo)
        ...
        # Crear usuarios (completar con credenciales de prueba)
        ...
        # rellenar SongUser
        ...
```

Este fragmento muestra cómo definir la clase `Command` que *Django* reconoce como implementación de un comando nuevo a añadir a `manage.py`. El resto de la implementación debe completarse siguiendo las especificaciones descritas anteriormente.

Cómo implementar el incremento automático de la variable number_times_played

Existen dos enfoques comunes para ejecutar lógica adicional cuando se crea o modifica un objeto de modelo: sobrescribir el método `save()` o utilizar señales (*signals*). Ambos enfoques son válidos y su elección depende del estilo arquitectónico del proyecto. Si se desea mantener la lógica encapsulada y simple, sobrescribir `save()` puede ser suficiente. Sin embargo, si se busca una separación más clara de responsabilidades y una arquitectura más desacoplada, el uso de señales es preferible. En esta práctica os solicitamos que uséis `signals`. Podéis ver un ejemplo de uso en `https://medium.com/django-unleashed/step-by-step-guide-to-django-signals-using-pre-save-post-save-pre-delete-and-post-delete-c74d8b1b929e`. En resumen, debéis utilizar una señal del tipo `post_save` para detectar cuándo se ha creado un nuevo objeto `SongUser` y, en respuesta, actualizar el campo `number_times_played` del objeto `Song` correspondiente.

2.1. Test

Con el fin de garantizar la integridad y el correcto funcionamiento de los modelos, se han desarrollado una serie de pruebas automatizadas utilizando el framework de pruebas de *Django*.

Las pruebas para el modelo `Song` incluyen:

- Verificación de la creación de instancias válidas con todos los campos requeridos.
- Comprobación de la representación en cadena del modelo (`__str__`).
- Validación de los valores permitidos para los campos `language` y `category`.
- Comprobación de que la variable `number_times_played` se autoincrementa.

Las pruebas para el modelo `SongUser` incluyen:

- Creación de relaciones válidas entre usuarios y canciones.

- Verificación de los valores por defecto en los campos `correct_guesses` y `wrong_guesses`.
- Comprobación de la representación en cadena del modelo.
- Prueba de integridad referencial: se intenta crear una instancia con un `song_id` inexistente, lo cual debe generar un error de integridad (`IntegrityError` o `ObjectDoesNotExist`).

Todas las pruebas están integradas en el sistema de pruebas automatizadas del proyecto y se ejecutan mediante el comando `python manage.py test song_models` (el fichero `song_models.py` se encuentra en el repositorio). Se recomienda su ejecución periódica durante el proceso de implementación del código.

Tanto en los test como en la cobertura (véase sección siguiente) ignorar fallos en el código que no hayáis creado vosotros.

Cobertura de Código

Para evaluar la efectividad de las pruebas implementadas utilizad la herramienta `coverage.py`, que permite medir qué partes del código fuente son ejecutadas durante la ejecución de los test. Se recomienda mantener una cobertura superior al 90 % y revisar periódicamente los informes generados para identificar posibles áreas no cubiertas por las pruebas.

Antes de ejecutar las pruebas, es recomendable eliminar los datos de cobertura anteriores para asegurar que el informe generado sea limpio y actualizado. Esto se puede hacer con el siguiente comando:

```
coverage erase
```

Luego, para ejecutar las pruebas junto con el análisis de cobertura, se puede utilizar:

```
coverage run manage.py test && coverage report -m
```

Si la cobertura del código que hayáis escrito o modificado vosotros es inferior al porcentaje requerido añadid todos los test necesarios para que esto ocurra. Guardad estos test en un fichero llamado `test_additional.py`.

3. Implementación del API REST (tiempo sugerido: la mitad de la práctica)

Todos los URLs descritos a continuación se asume que van precedidos por el prefijo `http://host/api/v1`.

3.1. Usuarios

Antes de describir el API específico del proyecto *songproject*, comenzaremos por implementar la gestión de usuarios, una funcionalidad esencial en muchas aplicaciones. Diseñaremos un sistema reutilizable y desacoplado del cliente, a diferencia de la práctica 1, donde se utilizaba *Django* tanto para el cliente como para el servidor.

En nuestra aplicación, tras el proceso de autenticación (login), se generará un token que deberá incluirse en cada solicitud al API como método de autenticación. Para ello, utilizaremos el módulo *Djoser*, específicamente la autenticación basada en tokens mediante `rest_framework.authtoken`.

La documentación oficial de *Djoser* está disponible en: `https://djoser.readthedocs.io/en/latest/index.html`. De todos los servicios que ofrece, utilizaremos únicamente los siguientes:

- /token/login/
- /token/logout/
- /users/me (para los test)

A continuación, se describe el comportamiento esperado al interactuar con el API desde un navegador:

1. `http://host/api/v1/token/login/` → Permite enviar el nombre de usuario y la contraseña para obtener un token. Este endpoint no requiere autenticación previa.
2. `http://host/api/v1/token/logout/` → Invalida el token del usuario autenticado, impidiendo su uso posterior.

3. `http://host/api/v1/users/me/` → devuelve información sobre el usuario "logeado".

Por defecto, todos los endpoints relacionados con *Djoser* requieren autenticación previa, excepto `/token/login`.

Para verificar el correcto funcionamiento del API, se puede utilizar la herramienta `curl` como se describe a continuación. Tras ejecutar "makemigrations" y "migrate", el primer paso sería poblar la base de datos con:

```
python manage.py populate
```

Y luego ejecutar los siguientes comandos:

- Login del usuario alumnodb (el API devuelve un token)

  ```
  $ curl -X POST http://127.0.0.1:8000/api/v1/auth/token/login/\
  --data 'username=alumnodb&password=alumnodb'
  ```

  ```
  Respuesta esperada:

  {"auth_token": "b704c9fc3655635646356ac2950269f352ea1139"}
  ```

- Consultar detalles del usuario autenticado:

  ```
  # do not forget to update the token value
  $ curl -LX GET http://127.0.0.1:8000/api/v1/auth/users/me/ -H\
  'Authorization: Token b704c9fc3655635646356ac2950269f352ea1139'
  ```

  ```
  Respuesta esperada:

  {"email": "", "username": "alumnodb", "id": 1}
  ```

3.2. Test

Para validar la implementación de los servicios proporcionados por *Djoser*, se incluye una batería de pruebas en el archivo `test_djoser.py`. Estas pruebas deben considerarse como requisitos funcionales del proyecto.

Los test asumen la siguiente configuración en `settings.py`: listado 2:

Listado 2: Configuración que informa a `djoser` de que se van a usar tokens como medio de autenticación y de que, por defecto, todos los métodos que se implementen en la API no deben ser accesibles para usuarios no autentificados

```
# settings.py
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework.authentication.TokenAuthentication',
    ),
    'DEFAULT_PERMISSION_CLASSES': (
        'rest_framework.permissions.IsAuthenticated',
    ),
}
```

NOTA IMPORTANTE: Se recomienda seguir una estrategia de desarrollo guiado por pruebas (TDD), ejecutando los test antes de implementar la funcionalidad.

3.3. Otros Endpoints

Usando `viewsets.ModelViewSet` o `viewsets.ReadOnlyModelViewSet` como clase base, se implementará la funcionalidad CRUD para los modelos de la base de datos. Para ello, se creará una segunda aplicación llamada `api`, y se registrarán automáticamente los endpoints en `api/urls.py` mediante routers:

Listado 3: ejemplo de router en `api/urls.py`

```
from rest_framework.routers import DefaultRouter
from .views import SongViewSet, SongUserViewSet
# from django.urls import path, include

router = DefaultRouter()
router.register(r'songs', SongViewSet, basename='songs')
router.register(r'songusers', SongUserViewSet,
    ↪ basename='songusers')

urlpatterns = []
urlpatterns += router.urls
```

En el URL `https://pytutorial.com/django-rest-framework-viewsets-and-routers-guide/` está accesible un tutorial sobre como crear un API REST a partir de modelos de *Django*. Igualmente en el URL `https://medium.com/django-unleashed/djangorestframework-drf-modelviewset-crud-with-sorting-search-and-pagination-c0ebb84a8d46` se describe como paginar un listado de modelos.

Detalles de los ViewSets

- `SongViewSet`: url → songs
  - Solo lectura (`ReadOnlyModelViewSet`).
  - Accesible sin autenticación.
  - Funcionalidades:
    - Listado paginado (3 canciones por página).
    - Detalles de una canción por ID. URL → songs/id
    - Obtener una canción aleatoria. Créese una acción nueva llamada `random` para implementar este requisito. Úsese el decorador "@action". (véase sección "Custom Actions in ViewSets" en `https://pytutorial.com/`

`django-rest-framework-viewsets-and-routers-guide/`). URL → songs/random

- Obtener las N primeras canciones con mayor `number_times_played`. Créese una acción nueva llamada `top` para implementar este requisito. Úsese el decorador "@action". URL → songs/top?n=3‘
- Obtener las canciones cuyo título contenga una cadena. Créese una acción nueva llamada `search` para implementar este requisito. Úsese el decorador "@action". URL → songs/search?tilte=cadena_busqueda‘

- `SongUserViewSet`: url → songusers
  - Implementa operaciones CRUD completas.
  - Uno de los atributos de `SongUserViewSet` es `user`. Puesto que este atributo solo es necesario cuando hay un usuario logeado se debe obtener a partir del objeto context: `context['request'].user` y el cliente no debe enviarlo al hacer una peticion `POST` de creación.

3.4. Testing y Cobertura

Para validar la API de juegos, se proporciona los test.

```
python manage.py test api.test_songuser_viewset
python manage.py test api.test_song_viewset.py
```

Además, se recomienda asegurar que el *coverage* de los archivos en la aplicación `api` sea cercano al 100 %. Si la cobertura del código que hayáis escrito o modificado vosotros es inferior al porcentaje requerido añadid todos los test necesarios para que esto ocurra. Guardad estos test en un fichero llamado `test_additional.py`.

4. Despliegue de la aplicación

El backend de la aplicación debe desplegarse en `render.com`. El URL del proyecto debe figurar en un fichero llamado `.env` situado en la raíz del proyecto. Igualmente

en este fichero se debe encontrar la URI de la base de datos creada en `neon.tech` y usada por la aplicación.

5. Test ejecutable en render.com

En el repositorio accesible en *Moodle* podéis encontrar el test `test_remote.py`. A diferencia de los test anteriores este puede ejecutarse en un servidor remoto. Este test asume que: (a) la aplicación está poblada usando `populate` y que existe la canción "Here in the real world" (los ficheros pertinente se encuentran disponibles en el repositorio concretamente en el directorio `Media`) y (b) existe un usuario llamado `alumnodb` con password `alumnodb`. El test realiza las siguientes acciones:

- registrarse usando el endpoint `token/login`.
- solicitar una canción aleatoria.
- solicitar las "n" canciones más populares
- buscar una canción
- crear un `SongUser`

A la terminación del test habría que comprobar que se ha creado una nueva entrada en la tabla `songuser` con los valores pertinentes y que el atributo `song.number_times_played` se ha incrementado en uno.

6. Resumen del trabajo a entregar

- Aseguraros de que vuestro código satisface los test proporcionados. No es admisible que se modifique el código de los mismos.
- Desplegad y probad la aplicación en `render.com` en modo de producción (`DEBUG=False`, `SECRET_KEY` y `DATABASE_URL` en una variable de entorno).

- Subid a *Moodle* el fichero obtenido al ejecutar el comando `zip -r ../assign3.zip .git` desde la raíz del proyecto. Recordad que hay que añadir (add) y enviar (commit) los ficheros a git antes de ejecutar el comando. Si queréis comprobar que el contenido del fichero zip es correcto lo podéis hacer ejecutando la orden: `cd ..; unzip assign3.zip; git clone . tmpDir; ls tmpDir`.

  IMPORTANTE: nos hará falta el URI para acceder a la base de datos creada en `neon.tech` y el URL en el que se ha desplegado la aplicación en `render.com` para evaluar esta práctica. Por favor, incluidlo en un fichero llamado `.env` que se encuentre en la raíz del proyecto. Si no se encuentra esta información la práctica estará suspendida. Asimismo, comprobar en *Render.com* que existe un usuario administrativo con username y password `alumnodb`.

7. Criterios de evaluación

Para obtener la clasificación de apto se deben satisfacer todos los criterios siguientes:

- La aplicación está desplegada en *Render.com* usnado el código entregado en *Moodle*.
- Las URLs en *Render.com* deben seguir el formato: `http://practica\_proyecto\_pareja\_grupo\_agno\_version.onrender.com` (en este caso: práctica = 3, proyecto = *songproject*, etc. `version` es una cadena arbitraria que el alumno puede elegir libremente.
- El test `test_remote.py` (sin modificaciones) se puede ejecutar apuntando a *Render.com* y se ejecuta sin reportar ningún fallo. Tras ejecutar el test
- Al finalizar el test las tablas `songuser` y `song` se han actualizado adecuadamente.
- Existe un fichero `.env` en la raíz del proyecto con la información solicitada.

Nota: Entrega fuera de plazo →, se substraerá un punto de la nota del examen por cada día (o fracción) de retraso en la entrega.

Nota: El código usado en la corrección de la práctica será el entregado en *Moodle*. Bajo ningún concepto se usará el código existente en *Render.com*, *Github* o cualquier otro repositorio.

A. Song

Listado 4: Código que muestra la lista de idiomas y categorias ofrecidas por la aplicación.

```

class Song(models.Model):
    LANGUAGE_CHOICES = [
      ('EN', 'English'),
      ('ES', 'Spanish'),
      ('FR', 'French'),
      ('DE', 'German'),
      ('IT', 'Italian'),
      ('PT', 'Portuguese'),
      ('JA', 'Japanese'),
      ('ZH', 'Chinese'),
    ]

    CATEGORY_CHOICES = [
        ('POP', 'Pop'),
        ('ROCK', 'Rock'),
        ('JAZZ', 'Jazz'),
        ('HIPHOP', 'Hip-Hop'),
        ('CLASSICAL', 'Classical'),
        ('REGGAE', 'Reggae'),
        ('LATIN', 'Latin'),
        ('KPOP', 'K-Pop'),
        ('COUNTRY', 'Country'),
        ('BLUES', 'Blues'),
        ('FOLK', 'Folk'),
        ('ELECTRONIC', 'Electronic'),
        ('R&B', 'R&B'),
        ('SOUL', 'Soul'),
        ('METAL', 'Metal'),
        ('PUNK', 'Punk'),
```

```
        ('ALTERNATIVE', 'Alternative'),
        ('INDIE', 'Indie'),
        ('GOSPEL', 'Gospel'),
        ('WORLD', 'World␣Music'),
    ]
```